BASES DE DATOS (TEORÍA RELACIONAL)

Tabla 1.1 FLIGHTS (airline schedule) RELATION

NUMBER	FROM	TO	DEPARTS	ARRIVES
83	JFK	O'Hare	11:30 a	1:43 p
84	O'Hare	JFK	3:00 p	5:55 p
109	JFK	Los Angeles	9:50 p	2:52 a
213	JFK	Boston	11:43 a	12:45 p
214	Boston	JFK	2:20 p	3:12 p

El formato de la relación es determinado por el conjunto de nombres de columnas: {NUMBER, FROM, TO, DEPARTS, ARRIVES}. A estos nombres les llamamos "nombres de atributos". Correspondiente a cada nombre de atributo, existe un conjunto de valores permisibles para la columna asociada. Este conjunto es llamado el "dominio del nombre del atributo". Cada fila en la relación es un conjunto de valores, uno del dominio de cada nombre de atributo. Las filas de esta relación se llaman "tuplas". Finalmente, existe un subconjunto del conjunto de nombres de atributos con la propiedad de que cada tupla puede ser distinguida por medio de ver solamente a los valores correspondientes a los nombres de atributos de dicho subconjunto. Tal subconjunto es llamado "Key para la relación". Para la relación en la tabla 1.1, {NUMBER} es una key.

1.2 Formalización de Relaciones.

Def: Un <u>esquema de una relación R</u> es un conjunto finito de nombres de atributos $\{A_1, A_2, \ldots, A_n\}$.

Correspondiente a cada nombre de atributo A_i, existe un conjunto D_i, $1 \leq i \leq n$, llamado el <u>dominio de A_i</u>.

También denotamos al dominio de A_i por $dom(A_i)$.

A los nombres de atributos los llamamos símbolos de atributos, o simplemente atributos, particularmente en el abstracto. <u>Los dominios son conjuntos arbitrarios, no vacíos y finitos o infinito-numerables.</u>

Sea $D = \bigcup_{i=1}^{n} D_i$. Una <u>relación r</u> en el esquema R es el conjunto definido de la siguiente manera: (Un conjunto finito de funciones $\{t_1, t_2, \ldots, t_r\}$ de $R \to D$)

$$r = \{ t : R \to D \mid \forall\, 1 \leq i \leq n \;\; t(A_i) \in D_i \}$$

Si todos los D_i's son finitos, entonces D es finito, y $\# D \leq \sum_{i=1}^{n} \# D_i$.

Más aún, $\# r = (\# D)^{\# R}$. En consecuencia, si $\# R = n$, entonces $\# r$ (o el número de tuplas posibles de la relación) es $(\# D)^n$.

* Más bien, la relación que consista de todas las tuplas $t : R \to D$ tiene cardinalidad $(\# D)^n$. En consecuencia, la cardinalidad de cualquier otra relación en el esquema $R[A_1 A_2 \cdots A_n]$ es menor o igual a $(\# D)^n$.

Ejemplo 1.1 En la tabla 1.1, el esquema de la relación

FLIGHTS = {NUMBER, FROM, TO, DEPARTS, ARRIVES}.

Los dominios para cada nombre de atributo pudieron ser:

1. dom(NUMBER) = El conjunto de int's disponibles por el SGDB (Número de filas disponibles en el disco duro).

2. dom(FROM) = dom(TO) = El conjunto de strings disponibles por el SGDB.

3. dom(DEPARTS) = dom(ARRIVES) = El conjunto de strings con formato fecha que admite el SGBD.

La relación en la tabla 1.1 tiene cinco tuplas. Donde una de ellas es definida como:

t(NUMBER) = 83, t(FROM) = 'JFK', t(TO) = 'O'Hare',

t(DEPARTS) = '11:30a', t(ARRIVES) = '1:43p'.

Usamos funciones en nuestro formalismo para evitar cualquier orden explícito en los nombres de atributos en el esquema de la relación. Como hemos visto, un cierto orden no añade información de contenido a una relación. Nosotros no queremos que las tuplas sean sucesiones de valores en cierto orden. En su lugar, una tupla es un conjunto de valores, uno para cada nombre de atributo en el esquema de la relación. Más aún, una tupla puede ser un multi-conjunto; esto es, un conjunto con duplicados, si dominios de dos nombres de atributos diferentes tienen elementos en común.

Dada una tupla t en una relación r en un esquema R, tiene sentido discutir el valor de t en el atributo A, alternativamente llamado "el valor-A de t".

Considerando a t como una tupla, el valor-A de t lo denotamos como $t(A)$. Puesto que t es una función, podemos restringir el dominio de t. Sea $X \subset R$. La notación usual para la función restricción a X es $t|_X$. Nosotros, en nuestro infinito conocimiento, confundiremos esto y escribiremos esta restricción como $t(X)$, y lo llamaremos "el valor-X de t". Técnicamente, $t(A)$ y $t(\{A\})$ son objetos diferentes, pero en orden de conservar las costumbres de confusión de la teoría de bases de datos relacionales, a menudo escribiremos a A como $\{A\}$. También eliminaremos la distinción entre $t(A)$ y $t(\{A\})$, aunque uno es un valor y el otro es un mapeo de A a ese valor. Asumimos que existe un valor λ tal que $t(\phi) = \lambda$ para cualquier tupla t.

Por tanto, $t_1(\phi) = t_2(\phi)$ $\forall\, t_1, t_2 \in r$.

Ejemplo. Sea $t: R \to D$ definida como: $t(NUMBER) = 83$, $t(FROM) = 'JFK'$, $t(TO) = 'O'Hare'$, $t(DEPARTS) = '11:30a'$, $t(ARRIVES) = '1:43p'$. Entonces, el valor-FROM de t es: 'JFK'. El valor-$\{FROM, TO\}$ de t es la tupla t' definida como $t'(FROM) = 'JFK'$, $t'(TO) = 'O'Hare'$.

Denotaremos a tal tupla como:

<'JFK' : FROM 'O'Hare' : TO>.

Hemos tratado a las relaciones como objetos estáticos. Sin embargo, las relaciones son una abstracción de una porción del mundo real, y esta porción puede cambiar con el paso del tiempo. Consideramos que las relaciones son variantes con el paso del tiempo, por lo que las tuplas pueden ser añadidas, eliminadas, o cambiadas. También asumimos que el esquema de la relación es invariante con el paso del tiempo. En consecuencia, cuando tratemos con una relación, debemos pensar en ella como una sucesión de relaciones en el sentido definido previamente, o, en algunos casos, como posibles sucesiones que la relación puede seguir, esto es, posibles estados que la relación puede ocupar.

1.3 Keys

Una key de una relación r en un esquema de relación R es un subconjunto $K = \{B_1, B_2, \dots, B_n\} \subseteq R$ de R tal que $\forall t_1, t_2 \in r \rightarrow \exists B \in K \ (t_1(B) \neq t_2(B))$

Obs: Recordemos que las tuplas son funciones, y dos funciones f y g son iguales $f = g$ ssi $Dom f = Dom g$, $Ran f = Ran g$ y $\forall x \in Dom f \ f(x) = g(x)$.
Ahora, las tuplas en una relación r siempre tendrán el mismo dominio y el mismo rango, por lo que serán diferentes ssi $\exists A \in R \ (t_1(A) \neq t_2(A))$.

Ejemplo. {NUMBER} es una KEY.

{FROM, TO} es una KEY.

Nuestra convención para la notación de relaciones, esquemas y llaves es la siguiente:
Usar letras del principio del alfabeto en mayúscula para símbolos de atributos, letras del final del alfabeto en mayúscula para los esquemas de relación y letras en minúscula para relaciones.
Denotamos el esquema de una relación $R = \{A_1, A_2, \ldots, A_n\}$ por $R[A_1 A_2 \cdots A_n]$, o algunas veces $A_1 A_2 \cdots A_n$ cuando no nos importe nombrar el esquema.
Una relación r en el esquema R es escrito como $r(R)$ o $r(A_1 A_2 \cdots A_n)$. Para denotar la key de una relación, subrayamos los nombres de atributos en el esquema de la relación y en la relación.
La relación r en el esquema ABCD con AC como key se escribe $r(\underline{A}B\underline{C}D)$; el esquema se escribe como $R[\underline{A}B\underline{C}D]$.

Ej: El esquema de la relación FLIGHTS de la tabla 1.1 se escribe como:

FLIGHTS [<u>NUMBER</u> FROM TO DEPARTS ARRIVES]

Si deseamos especificar más de una key para un esquema o relación, debemos enlistar las keys por separado, puesto que la notación por subrayado no funcionará.

Las llaves explícitamente enlistadas con un esquema de relación son llamadas: "<u>designated keys</u>". Pueden haber otras keys que las que están enlistadas; estas son llamadas "<u>implicit keys</u>". Algunas veces distinguimos a una de las <u>designed keys</u> como la "<u>primary key</u>".

Nuestra definición de key es un poco general. Si $r(R)$ tiene una key K' tal que $K' \subseteq K \subseteq R$, entonces K también es una key para R. Pues para cualesquiera dos tuplas t_1 y t_2 en r, si $t_1(K') \neq t_2(K')$, entonces, ciertamente $t_1(K) \neq t_2(K)$. A continuación restringimos nuestra definición ligeramente:

<u>Definición 1.1.</u> (Key / Superkey). <u>Una key de una relación $r(R)$</u> es un subconjunto K de R tal que

$$\forall t_1, t_2 \in r \rightarrow (t_1 \neq t_2 \rightarrow (t_1(K) \neq t_2(K)) \wedge \sim \exists K' (K' \subset K \wedge$$

$$\forall t_1, t_2 \in r \rightarrow (t_1 \neq t_2 \rightarrow (t_1(K') \neq t_2(K'))).$$

Equivalentemente,

$$\forall t_1, t_2 \in r \rightarrow (t_1 \neq t_2 \rightarrow (t_1(K) \neq t_2(K)) \wedge \forall K' (K' \subset K \rightarrow$$

$$\exists t_1, t_2 \in r \; (t_1 \neq t_2 \wedge t_1(K') = t_2(K'))).$$

<u>Una superkey de una relación R</u> es un subconjunto $\hat{K}$ de R tal que $\exists K \subseteq R$ tal que K es key de R y $K \subseteq \hat{K}$.

La nueva definición de superkey es la misma que la primer definición de key. Aún usamos esa definición de key en las designed keys, esto es, una designed key puede ser una superkey.

Ejemplo. En la tabla 1.1, {NUMBER} es una KEY (y una superkey), pero {NUMBER, FROM} es una superkey pero no es una KEY.

Existen algunas sutilezas con las keys. Como mencionamos en la sección anterior, consideramos que las relaciones son variantes con el paso del tiempo. Para cualquier estado de la relación, podemos determinar las keys y super-keys de la relación. Diferentes estados de la relación pueden tener diferentes keys. Sin embargo, consideramos esquemas de relación invariantes con el paso del tiempo; nosotros deseamos que las keys especificadas con los esquemas de la relación tampoco varíen. Por tanto, en orden de determinar keys para un esquema de relación, buscamos entre todos los posibles estados que la relación en un esquema puede asumir. LAS KEYS DEBEN DE MANTENERSE COMO KEYS PARA CUALQUIER INFORMACIÓN PERMISIBLE.

Ejemplo. Llamemos a la relación en la tabla 1.1 "Sched". Consideremos la siguiente actualización:

ADD (sched; NUMBER = 117, FROM = Atlanta, TO = Boston, DEPARTS = 10:05p, ARRIVES = 12:43a)

Cuando se asume un cierto orden en los nombres de atributos, la versión corta siguiente es suficiente:

ADD (r; $d_1, d_2, \ldots, d_n$)

Ejemplo. La versión corta del ejemplo anterior es:

ADD (sched; 117, Atlanta, Boston, 10:05p, 12:43a)

La intención de la operación ADD es clara; añadir la tupla descrita a la relación especificada. El resultado de la operación puede que no esté de acuerdo con la intención por una de las siguientes razones:

1. La tupla descrita no se acopla al esquema de la relación especificada.
2. Algunos valores de la tupla no pertenecen a los dominios apropiados.
3. La tupla descrita coincide en un valor de una key con una tupla existente en la relación.

Ejemplo. En la tabla 1.1, {FROM, TO} es una key para la relación. Sin embargo, es posible que puedan haber dos vuelos con el mismo origen y el mismo destino. En consecuencia, puede existir un estado de la relación en el que {FROM, TO} ya no será una KEY. Por esta razón, uno no considera a {FROM, TO} como una KEY.

Nosotros estaremos principalmente interesados en keys y superkeys de esquemas de relaciones, pensando en términos de todos los estados permisibles de una relación en el esquema. Lo que es y lo que no es una key es ultimadamente cuestión de semántica.

ACTUALIZACIÓN DE RELACIONES. AHORA QUE TENEMOS relaciones, ¿qué podemos hacer con ellas? Como la hemos establecido, el contenido de una relación varía con el tiempo, entonces, debemos de considerar cómo alterar una relación. Supongamos que deseamos poner más información en la relación. Realizamos una operación "add" en la relación. Para una relación $r(A_1 A_2 \cdots A_n)$, la operación add tiene la forma:

$$ADD(r; A_1 = d_1, A_2 = d_2, \ldots, A_n = d_n)$$

En cualquiera de estos casos, consideramos

ADD(r; $d_1, d_2, \ldots, d_n$) que regrese a r sin hacerle cambios y de alguna manera indicar el error.

<u>Ejemplo</u>. Si sched es la relación en la tabla 1.1, entonces

ADD(sched; NUMBER = 117, FROM = Atlanta, TO = Boston, DATE = 4 March)

no está permitido por la razón ①.

La operación:

ADD(sched; NUMBER = 84, FROM = O'Hare, TO = JFK, DEPARTS = 25:15p, ARRIVES = 6:00p)

no está permitido por las razones ② y ③.

NUMBER = 84 y hay otra tupla tal que NUMBER = 84 y DEPARTS = 25:15p no es un valor permitido.

+<u>OBS</u>: En un SGBD, los dominios son los tipos de datos que permite el mismo SGBD, por lo que es útil saber los nombres, tamaños, rangos, y valores de los tipos de datos que ofrece el SGBD.

De hecho, no tenemos que dar tanta información para identificar de manera única la tupla a ser removida. Especificar los valores de alguna Key es suficiente. Si $K = \{B_1, B_2, \ldots, B_m\}$ es una Key, entonces podemos usar la forma

$$DEL(r; B_1 = e_1, B_2 = e_2, \ldots, B_m = e_m)$$

Ejemplo. Una versión más corta del ejemplo anterior es:

$$DEL(sched; NUMBER = 83) \quad o$$

aún más corto:

$$DEL(sched; 83)$$ Pues {NUMBER} es una Key de sched.

primary designada

El resultado de la operación delete es como se esperaba. La tupla especificada es removida de la relación, excepto cuando la tupla no es un elemento de la relación. En ese caso, la relación se mantiene sin cambios y un error de condición es señalado. No hay restricción en remover la última tupla de la relación; la relación vacía es permitida.

En lugar de añadir o eliminar toda una tupla, tal vez deseemos modificar parte de la tupla. La modificación es lograda con la operación "change". Para una relación $r(A_1, A_2, \ldots, A_n)$ tal que $\{C_1, C_2, \ldots, C_p\} \subseteq \{A_1, A_2, \ldots, A_n\}$, entonces la operación change toma la siguiente forma:

$$CH(r; A_1 = d_1, A_2 = d_2, \ldots, A_n = d_n; C_1 = e_1, C_2 = e_2, \ldots, C_p = e_p).$$

Si $K = \{B_1, B_2, \ldots, B_m\}$ es una key, entonces la obtenemos como:

$$CH(r; B_1 = d_1, B_2 = d_2, \ldots, B_m = d_m; C_1 = e_1, C_2 = e_2, \ldots, C_p = e_p)$$

Ejemplo: Para la relación sched en la tabla 1.1, podemos tener:

CH(sched; NUMBER = 109, FROM = JFK, TO = Los Angeles, DEPARTS = 9:50a, ARRIVES = 2:52a; DEPARTS = 9:40a, ARRIVES = 2:42a), con la versión corta:

CH(sched; NUMBER = 109; DEPARTS = 9:40a, ARRIVES = 2:4

La operación change es meramente una conveniencia. El mismo resultado puede ser obtenido de una operación delete seguida por un add. Por tanto, todos los posibles errores para add y delete se aplican a la operación change: la tupla especificada no existe (delete), los cambios tienen un formato erróneo (1:- add), o usar valores fuera del dominio apropiado (2:- add) o la tupla cambiada tiene el mismo valor en una key que una tupla existente en la relación (3:- add).

Ejemplo. Efecto de aplicar las operaciones

1:- ADD (sched; 117, Atlanta, Boston, 10:05p, 12:43a)

2.- DEL (sched; FROM = JFK, TO = O'Hare, DEPARTS = 11:30a)

3.- CH (sched; NUMBER = 109; DEPARTS = 9:40p, ARRIVES = 2:42a)

es (sin orden):

NUMBER	FROM	TO	DEPARTS	ARRIVES
84	O'Hare	JFK	3:00p	5:55p
109	JFK	Los Angeles	9:40p	2:42a
213	JFK	Boston	11:43a	12:45p
214	Boston	JFK	2:20p	3:12p
117	Atlanta	Boston	10:05p	12:43a